NBA FRONT OFFICE LAB

Budget Contender Engine - Development Roadmap

A staged plan for building a data-driven NBA roster construction workspace focused on competitive performance, payroll efficiency, roster fit, and championship-level outcomes.

PROJECT VISION

Build a clean front-office workspace that can identify high-value NBA players, analyze historical team success, construct rosters under financial constraints, and eventually estimate playoff and championship outcomes without reducing roster building to simple points-per-dollar calculations.

Prepared as a build roadmap for implementation with Claude Code

1. Executive Summary

The NBA Front Office Lab is envisioned as a research and roster-construction application for answering a practical question: how can a team assemble a highly competitive roster - potentially at championship level - while operating within a constrained payroll budget?

The system should combine current player and team statistics, contracts, salary-cap rules, historical team outcomes, lineup and role fit, optimization, and eventually probabilistic modeling. The development path intentionally begins with trustworthy analytics before introducing roster scoring, optimization, or predictive claims.

Core principle

Do not ask only, "Which players have the best stats for the least money?" Ask, "Which combination of players creates the most championship-relevant basketball value under our financial constraints?"

Roadmap at a Glance

Version	Focus	Primary Outcome
V0	Project Foundation	Architecture, data contracts, season rules, testing foundation
V1	NBA Analytics Workspace	Current player/team stats, contracts, manual roster builder, clean UI
V2	Winning Benchmarks	Historical playoff and championship thresholds, era-aware analysis
V3	Player Value Engine	Basketball value, contract efficiency, surplus value, archetypes
V4	Roster Fit Engine	Complementary skills, redundancy, weaknesses, role coverage
V5	Budget Roster Optimizer	Generate strong rosters subject to payroll and roster constraints
V6	Contender Probability Model	Estimate playoff-round and championship outcomes
V7	Scenario Simulator	What-if signings, trades, injuries, development, Monte Carlo seasons
V8	Advanced Front Office Intelligence	Trades, aging curves, draft value, multi-year planning

2. Version 0 - Project Foundation

Before building a large feature set, establish clear architectural boundaries so data ingestion, analytics, optimization, modeling, and the UI can evolve independently.

NBA + Contract + Cap Data
-> Ingestion -> Cleaning + Normalization -> Storage
-> Analytics -> Player Value / Team Fit
-> Optimization -> Prediction / Simulation -> Front Office UI

Season-Specific Rules

Salary-cap rules and thresholds change by season, so financial rules should be stored as configuration rather than embedded throughout Python code.

configs/seasons/
2024-25.yaml
2025-26.yaml
2026-27.yaml

As a current reference point, the NBA announced the 2026-27 salary cap at $164.961M, the luxury-tax line at $200.428M, the first apron at $209.015M, and the second apron at $221.686M. These values should be source-verified when used and treated as season configuration, not permanent constants.

Foundational Engineering Rules

- Provider adapters retrieve data; processing modules clean and normalize it.
- Raw source data should remain immutable whenever practical.
- Analytics own calculations; the UI should not recalculate metrics.
- Optimization should consume tested analytics rather than duplicate formulas.
- Predictive models should be evaluated out of sample before influencing roster recommendations.
- Season-specific cap and roster rules must be configurable.
- Every major module should have automated tests before the next milestone depends on it.

Initial Data Domains

Domain	Examples
Players	Identity, age, position, physical profile, availability
Player performance	Box score, shooting, creation, rebounding, defense, advanced metrics
Teams	Record, pace, ratings, point differential, playoff outcome
Contracts	Salary, guarantees, options, years remaining, salary as % of cap

Domain	Examples
Cap rules	Cap, tax line, aprons, roster and transaction constraints
Lineups	Five-man combinations, minutes, offense, defense, net rating
Historical outcomes	Seeds, playoff rounds, Finals appearances, champions

VERSION 1

3. Version 1 - NBA Analytics Workspace

Version 1 should be a complete product by itself. It answers: "What is happening in the NBA right now, and how do players and teams compare?"

Core Data Capabilities

- Current player statistics and game logs.
- Current team statistics and standings.
- Advanced offensive and defensive metrics.
- Current contracts and future salary commitments.
- Season-specific salary-cap thresholds.
- Historical team records and playoff outcomes.

Player Profile

Category	Example Fields
Profile	Age, position, height, weight, team
Availability	Games played, minutes, starts, recent availability
Scoring	Points, FG%, 3P%, FT%, eFG%, TS%
Creation	Assists, assist rate, usage, turnover rate
Rebounding	Total, offensive, defensive, rebound rate
Defense	Steals, blocks, defensive indicators, role context
Impact	Offensive rating, defensive rating, net rating, on/off metrics
Contract	Salary, cap percentage, years remaining, guarantees, options

Important Financial Metric

Use salary as a percentage of the cap

A $25M player means something different under a $100M cap than under a $165M cap. Cap-normalized salary improves comparisons across seasons and is more useful for historical value analysis.

Scoring and Defensive Benchmarks

The system should calculate raw points scored and allowed because they are intuitive, but also possession-adjusted efficiency so teams playing at different tempos can be compared fairly.

Point Differential = **Points Per Game - Opponent Points Per Game**

Offensive Rating = (Points / Possessions) x 100

Net Rating = Offensive Rating - Defensive Rating

The workspace should display both raw scoring and efficiency-based metrics rather than treating a specific points-per-game threshold as universally meaningful.

FRONT OFFICE WORKBENCH

4. User Interface - Front Office Workbench

The clean UI should exist from Version 1. Streamlit is a practical first implementation because it keeps the application Python-first while supporting interactive tables, filters, charts, state, and rapid iteration.

Version 1 Navigation

Page	Purpose
League Overview	Current league landscape, standings, team efficiency, payroll context
Player Explorer	Search, filter, compare, and inspect individual players
Team Explorer	Analyze current teams, rotations, efficiency, and payroll
Contract Explorer	Review salaries, options, guarantees, and cap percentages
Analytics Lab	Run comparisons, historical filters, and custom calculations
Team Builder	Manually assemble a roster and see payroll plus team metrics
Data Health	Verify freshness, missing data, failed updates, and source status

Team Builder - V1 Experience

MY ROSTER		
PG	Player A	$18.4M
SG	Player B	$11.2M
SF	Player C	$28.1M
PF	Player D	$14.7M
C	Player E	$22.0M
Bench		$41.8M
Total Payroll		$136.2M

In Version 1, the Team Builder does not need to optimize automatically. It should let the user assemble players, track financial totals, and view aggregated team strengths so the interaction model is established before optimization is introduced.

UI Design Principles

- Persistent global filters for season, team, player, and comparison window.
- Headline KPIs at the top of each page; detailed analysis below.
- Clear separation between measured statistics, modeled estimates, and user assumptions.
- Interactive charts for exploration and static export charts for reports.
- Data freshness and source status visible from the UI.
- Advanced settings hidden behind expandable controls to keep the workspace clean.

VERSION 2

5. Version 2 - Winning and Championship Benchmarks

Version 2 asks: "Historically, what does a successful NBA team actually look like?" The objective is to quantify the characteristics associated with making the playoffs, advancing through rounds, reaching the Finals, and winning a championship.

Team-Season Research Dataset

- Wins, losses, win percentage, conference, and seed.
- Points scored and allowed.
- Offensive, defensive, and net ratings.
- Pace and point differential.
- Playoff qualification and round reached.
- Conference Finals, NBA Finals, and championship outcomes.

Probability Instead of Magic Thresholds

The workspace should avoid claims such as "50 wins makes a contender." Instead, calculate historical frequencies and conditional probabilities.

Example research output (illustrative only)

38 wins -> Playoff / Play-In probability: X%
42 wins -> Y%
46 wins -> Z%
50 wins -> ...
55 wins -> ...

Era-Aware Analysis

Raw scoring from different NBA eras should not be compared without context. The system should support modern-era filters and league-relative metrics.

Relative Offensive Strength = **Team Offensive Rating - League Average Offensive Rating**
Relative Defensive Strength = League Average Offensive Rating - Team Defensive Rating

Users should be able to filter all history, the last 5 or 10 seasons, the modern era, the Play-In era, or specific seasons.

VERSIONS 3 AND 4

6. Version 3 - Player Value Engine

Version 3 asks: "Which players provide the most basketball value for their salary?" The answer should not be based on points per dollar alone.

Offensive Impact + **Defensive Impact** + **Efficiency** + **Creation**
+ Shooting + Rebounding + Ball Security + Availability
+ Role Versatility + Age + Playoff Performance
-> Basketball Value -> Contract Cost -> Surplus Value

Potential Player Archetypes

Archetype	Role Example
Primary Creator	High-volume initiator who creates efficient offense for self and teammates
Secondary Creator	Can run offense without needing full-time primary usage
Movement Shooter	Creates spacing through off-ball shooting and movement
Point-of-Attack Defender	Defends lead guards and initiators at the perimeter
Switchable Wing	Multi-position defender with useful offensive connectivity

Archetype	Role Example
Rim Protector	Deters and contests shots near the basket
Stretch Big	Provides frontcourt size while preserving floor spacing
Connector	Low-usage player who passes, screens, cuts, and maintains flow

Scoring caution

Any composite player score should be empirically validated. Weights must not be presented as objective merely because they produce a clean 0-100 score.

7. Version 4 - Roster Fit Engine

Version 4 asks: "Do these players actually make sense together?" It evaluates complementary skills, missing roles, and redundancy rather than treating a roster as the sum of independent player ratings.

Area	Fit Questions
Offense	Creation, shooting, rim pressure, passing, turnover control
Defense	Perimeter defense, interior defense, rim protection, switching
Possession battle	Rebounding, turnovers, free-throw generation
Structure	Size, athleticism, positional flexibility, lineup balance
Depth	Bench creation, backup bigs, reserve defense, injury resilience
Usage	Can stars and high-usage players coexist without role conflict?

VERSION 5

8. Version 5 - Budget Roster Optimizer

This is the first major optimization stage. The user defines a payroll budget and roster requirements, and the engine searches for high-quality rosters that satisfy the constraints.

User Constraints

Budget: $165M

Minimum requirements:

2 primary creators

4 reliable shooters

2 strong perimeter defenders

1 rim protector

1 backup center

Minimum projected Net Rating: +5

Optimization Objective

Maximize:

Projected Team Quality + Roster Fit + Contract Value

Subject to:

Total Salary <= Budget

plus roster, role, position, and applicable cap-rule constraints

Interactive Optimizer Controls

- Lock a player into every generated roster.
- Exclude specific players.
- Require one or more archetypes or roles.
- Set maximum player salary or age range.
- Choose a payroll mode: cap, tax line, first apron, second apron, or custom owner budget.
- Generate multiple distinct roster solutions instead of one "best" answer.

Why multiple rosters?

Optimization weights are assumptions. Showing several high-quality roster solutions makes trade-offs visible and reduces the temptation to present one mathematically convenient lineup as uniquely correct.

VERSION 6

9. Version 6 - Contender Probability Model

Only after a strong historical dataset and reliable team analytics exist should the project estimate postseason outcomes.

Model Outputs

Outcome	Example Output
Make Playoffs	Estimated probability

Outcome	Example Output
Win First Round	Estimated probability
Reach Conference Finals	Estimated probability
Reach NBA Finals	Estimated probability
Win Championship	Estimated probability

Potential Inputs

- Net rating, offensive rating, defensive rating, point differential, win percentage.
- Top-player quality and lineup depth.
- Roster age and availability.
- Shooting, rebounding, turnover control, and defensive role coverage.
- Payroll distribution and concentration of salary.
- Playoff experience and historical performance where measurable.

Class Imbalance Problem

There is only one champion each season. A dataset containing hundreds of team-seasons may contain only a few dozen championship examples. That makes a single champion/not-champion classifier vulnerable to class imbalance and overfitting.

Preferred hierarchy:

Make Playoffs -> Win Series -> Conference Finals -> NBA Finals -> Champion

The model should be evaluated out of sample and calibrated so displayed probabilities have empirical meaning. Model uncertainty should remain visible in the UI.

VERSIONS 7 AND 8

10. Version 7 - Scenario and Season Simulator

The simulator turns roster construction into an interactive what-if laboratory. Users can modify a roster and compare the expected effects on payroll, team strength, and postseason outlook.

Metric	Before	After
Payroll	$152M	$161M
Projected Wins	47	51
Net Rating	+3.8	+5.7
Shooting Score	78	86
Defense Score	82	80

Metric	Before	After
Playoff Probability	82%	93%
Title Probability	4%	8%

The values above are illustrative. In the actual application, outputs must come from tested models and the interface should identify them as estimates rather than guarantees.

Simulation Dimensions

- Healthy season vs. average availability vs. high-injury scenario.
- Expected player development vs. optimistic improvement vs. regression.
- Different lineup and minute-allocation assumptions.
- Thousands of Monte Carlo season simulations once the model is mature enough.

11. Version 8 - Advanced Front Office Intelligence

Capability	Purpose
Trade Analyzer	Compare incoming/outgoing value, fit, salary, and projected roster impact
Free Agent Finder	Identify affordable players who address specific roster weaknesses
Extension Analyzer	Compare extension cost with expected future contribution
Aging Curves	Estimate likely development and decline by archetype and age
Draft Value	Incorporate rookie-scale contracts and expected contribution
Future Cap Planning	Project multi-season payroll and flexibility
Historical Comparables	Find similar players, teams, and roster structures

DATA MODEL

12. Core Database Design

The database should be normalized enough to preserve history while still supporting fast analytical queries. The following tables provide a practical starting point.

Table	Purpose
players	Stable player identity and profile fields
player_seasons	Season-level player statistics and advanced metrics
player_game_logs	Game-level performance and availability

Table	Purpose
teams	Stable team identity
team_seasons	Season-level team performance and outcomes
team_games	Game-level team results
team_rosters	Player membership by team and season
contracts	Contract-level metadata, guarantees, options, start/end
contract_years	Salary by season and cap percentage
salary_cap_rules	Season-specific cap, tax, apron, and rule configuration
lineups	Five-player lineup identities
lineup_stats	Minutes and performance for lineups
playoff_results	Series and postseason advancement
player_archetypes	Role classification outputs
player_value_scores	Versioned player value calculations
custom_rosters	User-created roster scenarios
roster_players	Members of each custom roster
model_predictions	Versioned postseason probability outputs
simulation_results	Scenario and Monte Carlo results

Contract-Year Fields

player_id
season
salary
salary_cap_pct
guaranteed_amount
years_remaining
player_option / team_option
contract_start / contract_end

Version everything that changes

Player value scores, model predictions, archetype assignments, cap rules, and simulation assumptions should carry version or season metadata so results can be reproduced later.

CLAUDE CODE BUILD PLAN

13. Claude Code Milestones

The project is ambitious enough that Claude Code should build it in bounded milestones rather than receiving one prompt to "build the championship optimizer." Each milestone should end with tests, data validation, UI review where relevant, an architecture review, and a Git commit.

#	Milestone	Deliverable
1	Repository and architecture	Create package structure, configuration, test foundation, documentation
2	NBA data ingestion	Players, teams, games, standings, advanced statistics
3	Contracts and salary-cap data	Contracts, season rules, cap-normalized salaries
4	Database and repositories	Persistent storage and clean data-access boundaries
5	Player analytics	Reusable player calculations and comparison tools
6	Team analytics	Team efficiency, point differential, pace, historical outcomes
7	Front Office UI foundation	Navigation, filters, player/team explorers, Team Builder
8	Historical contender analysis	Playoff/championship benchmark research
9	Player value engine	Validated value features, surplus value, archetypes
10	Roster fit engine	Fit, redundancy, role coverage, weaknesses
11	Roster optimizer	Budget and roster constraints, multiple candidate rosters
12	Playoff/championship model	Calibrated postseason probability models
13	Simulator	Scenario and Monte Carlo analysis
14	Full-system audit	Architecture, leakage, tests, UI, documentation, performance

Completion Gate for Every Milestone

- Targeted automated tests pass.
- Relevant data-quality checks pass.
- UI workflow is visually reviewed if the milestone affects the interface.

- No unrelated architectural boundary was weakened.
- Changes are documented and committed before the next milestone begins.

FIRST RELEASE TARGET

14. Recommended Version 1.0 Scope

Release name

NBA Front Office Lab v1.0 - League and Roster Analytics

Version 1.0 should feel complete even if no predictive model exists yet. That gives the project a portfolio-worthy stopping point and creates a reliable foundation for later optimization and modeling.

Version 1.0 Acceptance Criteria

- Current player and team statistics can be refreshed and inspected.
- Player contracts and season-specific cap information are available in the workspace.
- Player and team profiles expose consistent, tested metrics.
- The workspace compares points scored/allowed and possession-adjusted efficiency.
- A clean Player Explorer and Team Explorer are usable from the UI.
- The manual Team Builder tracks roster membership, payroll, cap percentage, and aggregated team indicators.
- Data Health displays freshness, failed updates, missing data, and source status.
- Historical team data is stored in a way that supports Version 2 benchmark research.
- All major analytics functions have automated tests.
- No predictive championship claim is presented before the historical benchmark and modeling stages exist.

The First Research Question for Version 2

What statistically separates ordinary teams from contenders?

Once Version 1 data is trustworthy, the next stage should measure how winning teams differ in point differential, efficiency, pace, roster composition, payroll distribution, and other characteristics - while accounting for era and sample size.

Long-Term Product Vision

The end-state product is a front-office research workbench where a user can inspect the league, identify undervalued players, assemble rosters, understand cap constraints, evaluate roster fit, generate budget-conscious alternatives, simulate changes, and review transparent estimates of postseason outcomes.

Reliable data -> Tested analytics -> Value + fit -> Optimization

-> Calibrated probability models -> Scenario simulation
-> Better-informed roster construction decisions

REFERENCE NOTES

15. Reference Notes

This roadmap is primarily a product and engineering plan. Current league rules and provider capabilities should be re-verified when implemented. The following sources were referenced in the planning discussion for current cap/CBA context and possible statistics access.

Reference	Use in Roadmap
NBA salary cap announcement	2026-27 salary cap, luxury tax, first apron, and second apron thresholds. Source: NBA.com / NBA release.
NBPA Collective Bargaining Agreement	Current CBA framework and agreement duration. Source: National Basketball Players Association.
nba_api project	Community-maintained Python client for NBA.com statistics and live endpoints. Source: github.com/swar/nba_api.
Basketball Reference contracts	Example public contract reference source; automated-use permissions should be checked before building a scraper. Source: basketball-reference.com/contracts/players.html.

Important

Performance models, value scores, optimizer outputs, and championship probabilities should be treated as analytical estimates. They should be validated against historical and out-of-sample results and should never be presented as guarantees.